Sri Lanka Institute of Information Technology

Lab Submission
<05>

<IT25101980>

<N.S.K Ariyawansha>

AI & ML | IT2011

B.Sc. (Hons) in Information Technology

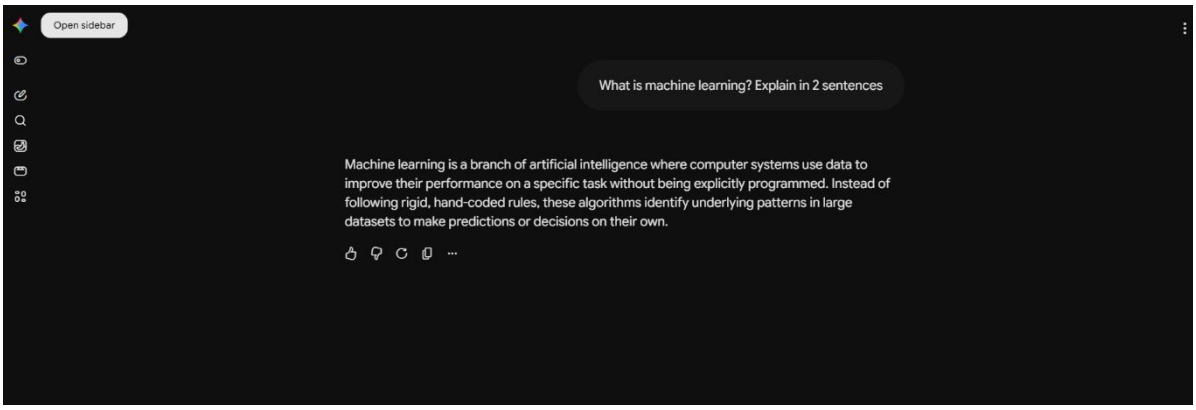
1)

2)

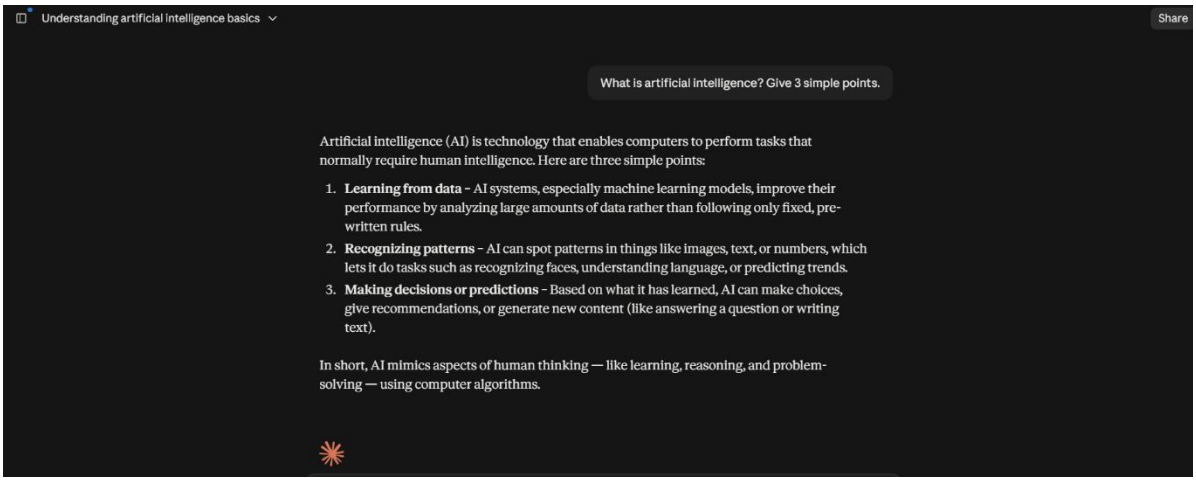
My favourite LLM is ChatGPT-5.6 Sol (max)

Its current rank is 5

Gemini



Cluade



3)

1.Deepseak

As a university student, you are in a unique position to use Generative AI not as a *crutch* that does your work for you, but as a **24/7, patient, Socratic tutor** that adapts to your learning style.

However, **Generative AI is notoriously bad at math** and **hallucinates code**. To use it effectively for STEM subjects, you must use **specific**

prompting strategies.

Here is your subject-by-subject playbook for using GenAI to actually *learn*.

---

1. Probability and Statistics (The Math Trap)

The Problem: AI often gets complex calculations wrong. If you ask for the answer to a problem, it will confidently give you a wrong number.
The Strategy: Use AI for *conceptual understanding* and *error checking*, NEVER for initial calculation.

- **Prompt for the "Rubber Duck" Explanation:** *"Act as a statistics professor. Explain the Central Limit Theorem to me like I am a sophomore, but use a relatable analogy (like pulling gumballs from a machine). Then, explain why it fails if the population variance is infinite."*
- **Prompt for Step-by-Step Derivation (NO NUMBERS):** *"Do not solve this equation. Instead, walk me through the logical steps of performing a two-tailed t-test. At each step, tell me why we do that step before moving to the next."*
- **Prompt for "What if" Scenarios:** *"I understand that $p < 0.05$ means reject the null. Give me 3 real-world scenarios where a p-value of 0.06 might actually be more actionable than a p-value of 0.04, based on context."*
- **The "Code to Check" Prompt:** *"I have written this Python script to calculate a binomial distribution. Do not rewrite my code. Just act as a linter and tell me if I have misused any statistical functions or variables."* **(Copy-paste your code).**

---

2. Database Management Systems (DBMS)

The Problem: AI writes SQL beautifully, but if you just copy-paste, you learn nothing about indexing, normalization, or query optimization.
The Strategy: Use AI as a **Query Optimizer** and **Normalization Coach**.

- **Prompt for Normalization Drills:** *"Give me a denormalized table with 5 columns full of redundancy. Do NOT give me the solution. Ask me to normalize it to 3NF. After I give you my answer, critique my foreign key placements and tell me where I broke atomicity."*
- **Prompt for Query Plan Visualization:** *"Here is my SQL query*

[paste query] and here is my table schema [paste schema]. Do not rewrite the query. Explain to me, in plain English, what a database engine would do step-by-step to execute this query. Point out where my 'WHERE' clause might cause a full table scan."

- **Prompt for ACID Drills:** *"Give me a tricky transaction scenario (e.g., a banking transfer) and ask me to write the SQL transaction using BEGIN, COMMIT, and ROLLBACK. After I write it, quiz me on what would happen if the power failed right after my second UPDATE statement."*
- **The "ERD to Schema" Reverse Prompt:** *"Describe a database for a library system in plain text. I will draw the Entity-Relationship diagram. After I describe my diagram to you, point out the many-to-many relationships I missed and suggest intersection tables."*

---

3. Software Engineering (The Design & Architecture)

The Problem: AI generates boilerplate code easily, but software engineering is about **trade-offs**, **scalability**, and **design patterns**—not just syntax.
The Strategy: Use AI to simulate a **Senior Engineer** during design reviews, not as a code generator.

- **Prompt for System Design Trade-offs:** *"I am designing a ride-sharing app. I am choosing between a monolithic backend and microservices. Do not tell me which is better. Give me a table of pros and cons specifically for my use case, focusing on deployment complexity and data consistency."*
- **Prompt for Design Pattern Recognition:** *"I have a problem where I need to create different types of 'Notifications' (Email, SMS, Push) based on user preferences. Suggest 3 different design patterns that could solve this. For each, tell me one scenario where that pattern would be a terrible choice."*
- **Prompt for Code Review (Crucial):** *"Act as a harsh tech lead. Here is my Java method [paste code]. Do not refactor it. Instead, point out 5 things wrong with it regarding SOLID principles, coupling, or potential null pointer exceptions. Give me hints, not the fix, so I can correct it."*
- **Prompt for Testing Strategies:** *"Here is a function that validates user input. Suggest 10 edge-case test inputs I should use for unit testing. Do not write the test code—just give me the inputs and tell*

me what expected outcome I should look for."

The "Golden Rules" for Using AI as a STEM Student

To make this work, you must enforce these 4 rules on yourself:

1. **The "No-Final-Answer" Rule:** Force the AI to explain *processes*, not provide *products*. If it gives you a direct answer to a stats problem, reply: *"Do not give me the number. Give me the equation I need to plug my variables into."*
2. **The "Socratic Backfire":** If the AI explains something, immediately ask: *"Now challenge my understanding. Ask me a question about what you just explained that would expose a common misconception."*
3. **The "Context Window" Trick:** Paste your entire lecture slides or textbook chapter into the prompt (if under the token limit) and say: *"Based only on this provided text, create a 10-question practice quiz. Do not use outside knowledge."* This restricts hallucinations to your specific curriculum.
4. **The Final Sanity Check: Never** submit AI-generated code or math without running it. For Stats, run the AI's suggested formula through a calculator. For DBMS, run the SQL on a test dataset. For Software Engineering, compile the code. The AI is a thought-partner; the compiler/calculator is the final authority.

Your go-to prompt for any subject: *"I am a student. I do not want the answer. I want to practice. Give me a problem, wait for my response, and then give me feedback on my approach without giving away the correct answer immediately."*

2. Grok

Generative AI serves as an always-available tutor, practice generator, explainer, and code/debug assistant for subjects like probability & statistics, DBMS, and software engineering—when used actively for understanding rather than passive copying.

Use it responsibly: treat outputs as starting points to verify against textbooks, lectures, or official docs; never submit AI-generated work as

your own; and focus on building genuine comprehension.

General Effective Techniques

- **Concept explanation and progressive deepening**: Ask for explanations at different levels (intuitive → formal → advanced). Follow up with "Explain the intuition," "Give the mathematical derivation," or "Compare this to [related concept]."

- **Active practice**: Request custom problems with solutions, then solve them yourself first and ask the AI to check your work or explain mistakes.

- **Socratic dialogue**: Instruct the AI to quiz you or guide you with questions instead of giving direct answers.

- **Visualization and analogies**: Ask for real-world analogies, step-by-step walkthroughs, or descriptions you can turn into diagrams/code.

- **Study aids**: Generate summaries, flashcards, mind maps (as text outlines), exam-style questions, or spaced-repetition schedules tailored to your syllabus.

- **Prompting tips**: Be specific ("Explain Bayes' theorem with a medical testing example and the formula"), provide context (your current knowledge level or course topics), and iterate ("That was too advanced—simplify" or "Now give a harder variant").

- **Verification habit**: Cross-check formulas, theorems, SQL syntax, design patterns, or proofs with reliable sources.

Probability and Statistics

Generative AI excels at making abstract math concrete and generating endless practice.

- Request clear explanations of core ideas (random variables, distributions, expectation, variance, conditional probability, hypothesis testing, confidence intervals, regression) with everyday examples (coin flips, medical tests, A/B testing).

- Ask for derivations or proofs broken into steps, then request variations or common pitfalls.

- Generate practice problems: “Create 5 problems on binomial and Poisson distributions with solutions and intermediate steps.” Solve them yourself, then compare.

- Explore simulations: Ask for Python/R code using libraries like NumPy, SciPy, or matplotlib to simulate distributions, Central Limit Theorem, or Monte Carlo methods, then run and modify the code yourself.

- Interpret results: Paste sample data or output and ask for statistical interpretation, assumption checks, or visualization suggestions.

- Connect theory to applications: “How is the law of large numbers used in machine learning?” or “Explain p-values without jargon, then with proper statistical language.”

This builds both intuition and technical fluency far faster than static textbooks alone.

Database Management Systems (DBMS)

AI is especially strong for query writing, schema design, and conceptual modeling.

- Schema and design help: Describe requirements and ask for ER diagrams (as text or Mermaid syntax), normalized relational schemas (up to 3NF/BCNF), or discussions of trade-offs (normalization vs. performance).

- SQL practice: Request example schemas + queries (SELECT with JOINs, subqueries, window functions, aggregation). Ask it to generate test data, then write and debug your own queries against them. “Explain why this query is slow and how to optimize it with indexes.”

- Core concepts: Get explanations of ACID properties, transactions, concurrency control (locking, MVCC), indexing (B-trees, hashing), query optimization, NoSQL vs. relational trade-offs, or distributed databases.

- Hands-on projects: Describe a mini-application (e.g., library system) and iteratively refine the schema, queries, and integrity constraints with AI feedback.

- Troubleshooting: Paste error messages or slow queries and request diagnoses plus corrected versions with explanations.

Always test generated SQL in a real DBMS (PostgreSQL, MySQL, SQLite) so you internalize the behavior.

Software Engineering

Use AI as a pair programmer, design critic, and process explainer while still writing and reviewing code yourself.

- Requirements and design: Feed informal requirements and ask for use cases, user stories, UML-style descriptions (class, sequence, activity diagrams in text/Mermaid), or architecture suggestions (layered, microservices, MVC). Critique the suggestions.

- Design patterns and principles: “Explain the Observer pattern with a concrete code example in [language], then show a situation where it would be overkill.” Cover SOLID, DRY, coupling/cohesion, etc.

- Coding and debugging: Describe a feature or paste incomplete code; ask for implementations, unit tests, or refactoring suggestions. Debug by describing symptoms or pasting stack traces—then implement the fix yourself.

- Process and lifecycle: Clarify Agile/Scrum/Kanban practices, version control workflows (Git branching strategies), testing pyramid (unit/integration/E2E), CI/CD basics, or code review checklists.

- Quality and maintenance: Request code reviews of your snippets focused on readability, edge cases, security, or performance. Generate test cases or ask how to measure complexity/metrics.

- Project support: Break a course project into milestones, generate boilerplate (with understanding), or explore trade-offs in technology choices.

The highest value comes from using AI to accelerate iteration while you retain ownership of design decisions and code quality.

Best Practices and Caveats

- Combine with traditional resources: lectures, textbooks (e.g., Ross for probability, Silberschatz for DBMS, Pressman or Sommerville for SE), official documentation, and hands-on tools (R/Python for stats, actual databases, IDEs and Git).

- Avoid over-reliance: If you only copy answers, exam performance and long-term retention suffer. Force yourself to solve, explain back, or implement before asking for the solution.

- Academic integrity: Follow your university's policies. Using AI for learning and practice is usually fine; submitting unattributed AI output is not.

- Limitations: AI can hallucinate formulas, outdated syntax, or suboptimal designs. Always verify critical details.

- Advanced use: Feed lecture notes or syllabus topics for personalized quizzes, or ask the AI to role-play as an examiner/code reviewer.

Start with focused sessions (e.g., "Help me master conditional probability today") and build a habit of active engagement. Over a semester this approach can turn difficult subjects into more manageable, even enjoyable, ones by providing instant, personalized feedback loops.

2. Qwen

Generative AI can be a transformative, 24/7 personalized tutor for a university student, provided it is used as a **learning accelerator** rather than a shortcut. Below is a comprehensive, subject-specific guide on how to leverage generative AI to master Probability and Statistics, Database Management Systems (DBMS), and Software Engineering, along with best practices to ensure academic integrity and deep understanding.

---

1. Probability and Statistics

Challenge: Abstract concepts, complex formulas, and connecting theory to real-world data.

How to Use Generative AI:

- **Socratic Concept Breakdown:** Ask the AI to explain complex theorems (e.g., Central Limit Theorem, Bayes' Theorem) using analogies tailored to your interests (e.g., sports, gaming, or finance).

- **Step-by-Step Problem Solving:** Instead of asking for the final answer, prompt the AI to guide you.

  - *Prompt:* "I'm trying to solve this hypothesis testing problem: [insert problem]. Don't give me the answer. Instead, ask me guiding questions one step at a time to help me figure out which test to use and how to calculate the p-value."

- **Synthetic Data Generation:** Practice statistical calculations by having the AI generate small, manageable datasets.

  - *Prompt:* "Generate a small CSV-formatted dataset of 20 students with variables: 'Study_Hours', 'Attendance_%', and 'Exam_Score'. Then, give me 3 questions to practice calculating mean, variance, and correlation on this data."

- **Code for Visualization:** Bridge theory and practice by asking the AI to write Python (Matplotlib/Seaborn) or R code to visualize concepts like normal distributions, confidence intervals, or regression lines.

---

2. Database Management Systems (DBMS)

Challenge: Moving from theoretical concepts (normalization, ACID) to practical implementation (SQL, schema design).

How to Use Generative AI:

- **Interactive Schema Design:** Practice translating real-world requirements into database structures.

  - *Prompt:* "Act as a database architect. I want to design a database for a university library system. Ask me 5 questions about the requirements. Based on my answers, help me draft an Entity-Relationship (ER) diagram and identify primary/foreign keys."

- **Guided Normalization Practice:**

  - *Prompt:* "Here is an unnormalized table with student course enrollments: [insert table]. Guide me step-by-step through converting this to 1NF, 2NF, and 3NF. Explain the functional dependencies at each step."

- **SQL Debugging and Optimization:** Paste your broken or slow SQL queries and ask for explanations, not just fixes.

  - *Prompt:* "Here is my SQL query to find the second-highest salary per department. It’s returning a syntax error. Can you explain what I did wrong and how the `WINDOW` function or `GROUP BY` should be applied here?"

- **Concept Clarification:** Use AI to demystify abstract concepts like indexing (B-Trees), transaction isolation levels, or concurrency control.

---

3. Software Engineering

Challenge: Bridging the gap between writing code and engineering robust, scalable, and maintainable systems.

How to Use Generative AI:

- **Design Pattern Tutor:** Learn when and how to apply design patterns (e.g., Singleton, Factory, Observer, Strategy).

  - *Prompt:* "Explain the difference between the Strategy and State design patterns using a simple Java example. Then, give me a real-world software scenario and ask me which pattern is more appropriate and why."

- **Code Review and Refactoring:** Treat the AI as a senior developer reviewing your pull requests.

  - *Prompt:* "Here is my Python code for a REST API endpoint. Review it for adherence to SOLID principles, potential security vulnerabilities (like SQL injection), and suggest improvements for readability."

- **Project Planning and Breakdown:** Use AI to practice Agile/Scrum methodologies.

  - *Prompt:* "I am building a task management web app. Help me break

this down into Epics and User Stories. Then, suggest a realistic tech stack and explain the trade-offs of your choices."

- **Mock Interviews:** Practice system design or behavioral interviews.

  - *Prompt:* "Act as a technical interviewer. Ask me a medium-difficulty system design question (e.g., design a URL shortener). Wait for my response, then critique my answer and ask follow-up questions about scalability and database choice."

Cross-Cutting Study Techniques

Regardless of the subject, these AI-powered strategies boost retention:

1. **Generate Custom Quizzes:** "Create a 10-question multiple-choice quiz on ACID properties in DBMS. Provide the answers and detailed explanations only after I submit my responses."

2. **Summarize Lecture Notes:** Paste your messy, rapid-fire lecture notes and ask: "Organize these notes into a structured study guide with key definitions, formulas, and a 'common pitfalls' section."

3. **Feynman Technique Practice:** Explain a concept to the AI and ask it to evaluate your understanding. "I'm going to explain how a B-Tree index works. Listen to my explanation, then point out any inaccuracies or gaps in my logic."

Best Practices & Pitfalls to Avoid

- **Beware of Hallucinations:** Generative AI can confidently state incorrect formulas, fake SQL syntax, or outdated software engineering practices. *Always cross-check critical information* with your textbook, official documentation, or professor.

- **Active vs. Passive Learning:** Do not just read AI-generated explanations. Force yourself to solve the problem, write the code, or draw the diagram first, then use AI to check your work.

- **Iterative Prompting:** The first response is rarely the best. Treat the AI

as a conversational partner. Ask follow-ups like, "Can you make that simpler?" or "Can you give me a counter-example?"

- **Academic Integrity:** Use AI to *learn how to do the work*, not to *do the work for you*. Never paste an AI-generated response directly into a graded assignment, exam, or project without fully understanding and modifying it. Check your university's specific AI policy.

Recommended Workflow for a Study Session

1. **Pre-study:** Ask AI to generate a 1-page overview of the topic you're about to learn in class to build a mental scaffold.

2. **During study:** Use AI to explain confusing textbook paragraphs or debug your practice code.

3. **Post-study:** Have AI generate a short quiz or flashcards to test your retention, and ask it to identify weak areas based on your answers.

By positioning generative AI as a **Socratic tutor, code reviewer, and study planner**, you can dramatically accelerate your mastery of these technically demanding subjects while building deeper, more resilient understanding.

I found all three AI tools useful for learning Probability and Statistics, DBMS, and Software Engineering. DeepSeek provided very detailed and practical suggestions, while Grok gave a clear and well-organized explanation. Qwen also provided useful examples and study techniques. In my opinion, DeepSeek produced the best result because its response gave practical ways to use Generative AI for learning and included useful examples of prompts.

4)

In [5]:
```
# Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a helpful assistant with deep knowledge in physics."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Explain how black holes are formed."}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

```
Black holes are formed when a massive star runs out of fuel and collapses in on itself during a supernova explosion. As t
he star collapses, the outer layers are blasted into space while the core collapses under its own gravity. If the core ha
s a mass greater than about three times that of the Sun, it will collapse into a black hole.

The core of the star becomes so dense that its gravity becomes incredibly strong, causing it to collapse to a point of in
finite density called a singularity. This singularity is surrounded by an event horizon, which is the point of no return
for anything that crosses it. Once an object or light crosses the event horizon, it can never escape the gravitational pu
ll of the black hole.

Black holes come in different sizes, with stellar-mass black holes being formed from the collapse of massive stars, and s
upermassive black holes found at the centers of galaxies, with masses millions or even billions of times that of the Sun.
```

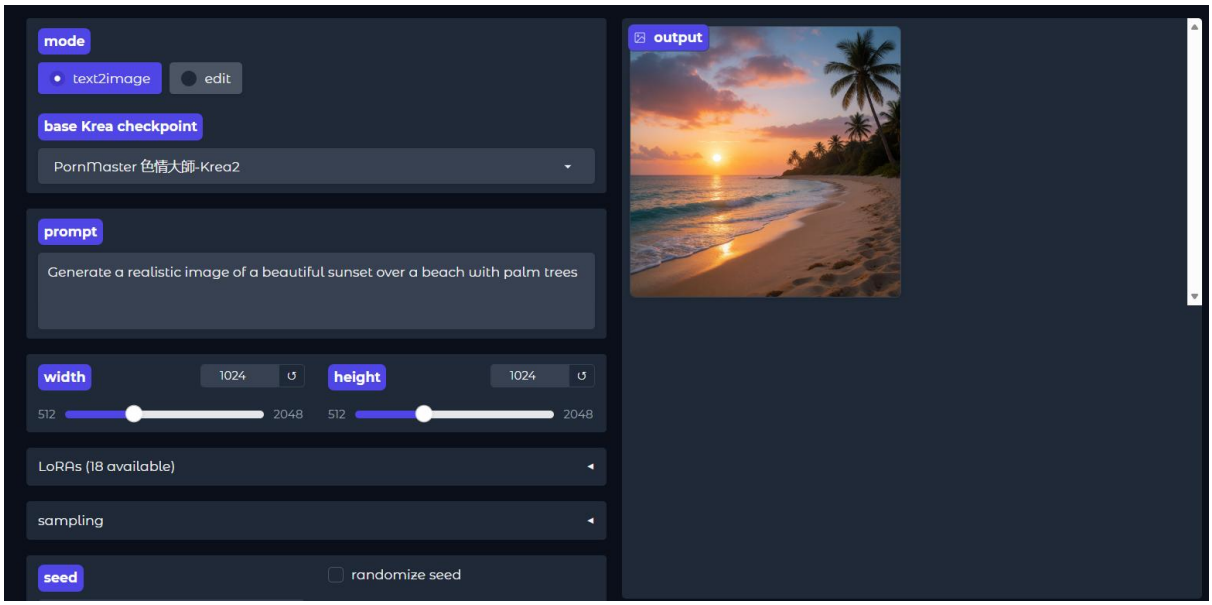
In [6]:
```
# Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are an expert event planner who gives simple and practical advice."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Help me plan my friend's birthday party. Give me 5 simple tips."}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

```
1. Set a Budget: First, determine how much you want to spend on the party. This will guide your planning process and help
you stay on track financially.

2. Choose a Theme: Pick a fun and festive theme for the party that suits your friend's personality. This will make it eas
ier to plan the decorations, activities, and food.

3. Create a Guest List: Make a list of all the people your friend would like to invite to the party. Consider the venue s
ize and your budget when finalizing the guest list.

4. Plan the Menu: Decide on the food and drinks you will be serving at the party. Consider if you will be cooking or orde
ring catering and make sure to accommodate any dietary restrictions or preferences.

5. Organize Entertainment: Plan some entertainment or activities for the party to keep guests engaged and create a fun at
mosphere. This could include games, music, or even hiring a performer.
```

In [10]:
```
# Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are an expert university study planner."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Create a simple study plan for a university student with 3 subjects."}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

```
Certainly! Here is a simple study plan for a university student with 3 subjects:

Subject 1: Mathematics
- Allocate 2 hours every morning for Mathematics practice and problem-solving.
- Review class notes and textbook material for 1 hour in the afternoon.
- Spend 1 hour in the evening reviewing past quizzes and tests.

Subject 2: Biology
- Set aside 1 hour every morning for reading and taking notes on Biology lectures.
- Spend 1 hour in the afternoon working on lab assignments or practical exercises.
- Review key concepts and diagrams for 30 minutes in the evening.

Subject 3: History
- Dedicate 1 hour every evening to reading History textbooks and supplementary materials.
- Take 30 minutes in the afternoon to summarize key points and create study guides.
- Review timelines and important dates for 30 minutes before bed.

Additionally, allocate time for breaks, physical activity, and adequate rest to ensure effective learning and retention.
Adjust the study plan as needed based on assignment deadlines, exams, and other commitments.
```

5)



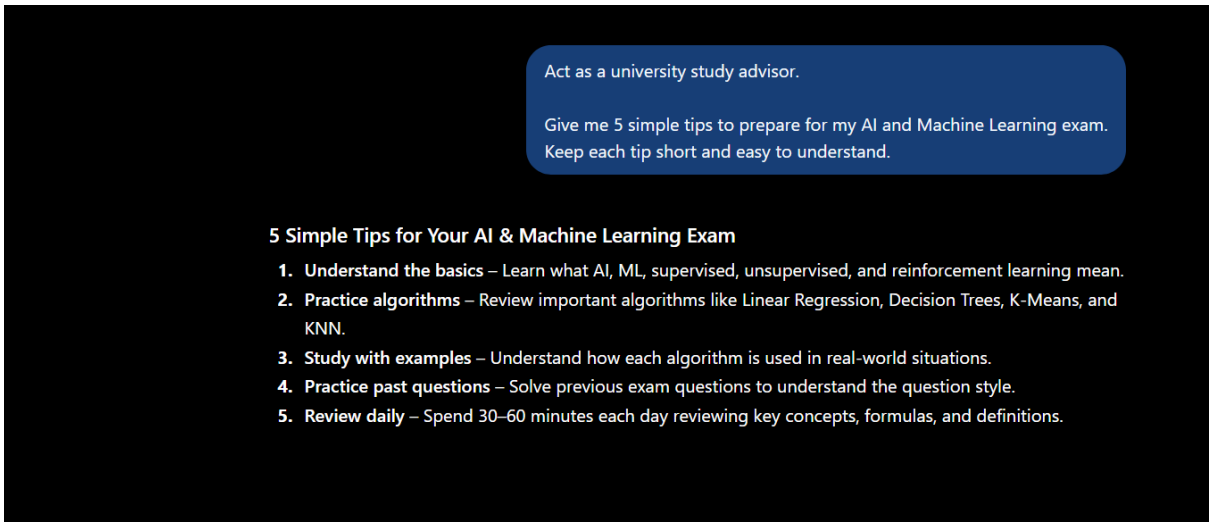

```
from transformers import pipeline

summarizer = pipeline("summarization")

text = """Hugging Face is a pioneering open-source AI platform
that empowers developers, researchers, and organizations to easily access and
deploy cutting-edge machine learning models for natural language processing,
computer vision, and beyond—democratizing AI and accelerating innovation through
a collaborative ecosystem of tools, datasets, and community-driven contributions."""

summary = summarizer(text, max_length=20)

print(summary)
```

```
No model was supplied, defaulted to sshleifer/distilbart-cnn-12-6 and revision a4f8f3e (https://huggingface.co/sshleifer/distilbart-cnn-12-6).
Using a pipeline without specifying a model name and revision in production is not recommended.
Device set to use cuda:0
Your min_length=56 must be inferior than your max_length=20.
[{'summary_text': ' Hugging Face is a pioneering open-source AI platform that empowers developers, researchers'}]
```

```
from transformers import pipeline

classifier = pipeline(
    "sentiment-analysis",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english"
)

text = "I really enjoy learning Artificial Intelligence."

result = classifier(text)

print(result)
```

```
config.json: 100% 629/629 [00:00<00:00, 34.8kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
model.safetensors: reconstructing file: 100% 268MB / 268MB, 25.1MB/s
model.safetensors: downloading bytes: 250MB, 23.0MB/s
Loading weights: 100% 104/104 [00:00<00:00, 2660.63it/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.62kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 12.8MB/s]
[{'label': 'POSITIVE', 'score': 0.9994668364524841}]
```

6)


Act as a university study advisor.
Give me 5 simple tips to prepare for my AI and Machine Learning exam. Keep each tip short and easy to understand.
5 Simple Tips for Your AI & Machine Learning Exam
1. Understand the basics – Learn what AI, ML, supervised, unsupervised, and reinforcement learning mean.
2. Practice algorithms – Review important algorithms like Linear Regression, Decision Trees, K-Means, and KNN.
3. Study with examples – Understand how each algorithm is used in real-world situations.
4. Practice past questions – Solve previous exam questions to understand the question style.
5. Review daily – Spend 30–60 minutes each day reviewing key concepts, formulas, and definitions.